

Poker Bots

Adam Ma, Ben Marler

Northeastern University

ma.ad@northeastern.edu, marler.b@northeastern.edu

Abstract

In this study, we sought to create two types of competitive poker playing agents through a combination of reinforcement learning and machine learning. Four fixed strategy opponents with clear weaknesses were developed to represent extreme versions of the four archetypes of poker players (loose passive, loose aggressive, tight passive, and tight aggressive). Four specialist DQN based agents were trained against each respective opponent in order to learn and exploit weaknesses in their strategies. A generalist Actor Critic based agent was also trained against the entire pool of opponents, learning to recognize a weakness and then exploit it once more experience was gained. Matchups were created between each agent and each opponent after training, in order to estimate the profitability of each agent's learned exploitative strategies against each player archetype.

Code — <https://github.com/adammotts/PokerBots>

Introduction

Back in the last 20th century, Texas Hold'Em held a lot of mystery and intrigue. Many top players gained popularity through fancy play and impressive bluffs, which was extensively sensationalized through television. Students of the game needed to figure things like "pot odds" and "equity" out using simple math and heuristics, and the concept of optimal strategy was unknown. This trope shifted dramatically with the rise of "solvers". The emergence of poker theory gave way to the utilization of counterfactual regret minimization, which is the leading methodology for solving games of imperfect information such as poker. Following massive amounts of computation and extensive development, suddenly, anyone with access to the internet could find out the "game theoretically optimal" (GTO) action in any given state. Poker lost some of its mystery and intrigue as top players who relied on psychology and creativity were replaced by well-studied intellectuals who memorized solver outputs. Fortunately, poker isn't even remotely close to being "boring".

While GTO is quite literally "optimal", there are many instances in which it is much more profitable to deviate from its doctrine. Consider the game of rock, paper, scissors.

GTO strategy in rock, paper, scissors is to play all three actions at an equal and completely random frequency. Now consider a drunkard who only plays rock, no matter the time of day, opponent, or state of affairs. GTO strategy against this opponent would only draw in the long run, creating an equal number of wins, losses, and draws. But "optimal", exploitative strategy would be to play paper at a 100% frequency, generating wins as long as both players are willing to continue playing. This example is reflective of poker as well: real world opponents are not playing optimally; in fact, they have many flaws. A solver may be able to exploit these gradually in the long run, but a human with the proper adjustments and deviations from theory may be able to exploit these violently and lucratively in a reasonably short span of time.

All of the aforementioned history and commentary serves as the motivation for this paper: to develop poker playing agents with the help of reinforcement learning and machine learning based methods that can learn to deviate from optimal strategy in order to exploit suboptimal player archetypes.

Related Work

A considerable amount of literature exists regarding poker bot development, spanning from early game-theoretic models to modern deep reinforcement learning. For the scope of our project, three specific areas of prior research are most relevant: GTO solvers, applying RL to poker environments directly, and building opponent models to exploit specific playstyles.

Game-Theoretic Foundations

Counterfactual Regret Minimization (CFR) has largely dominated the computer poker space. By iteratively minimizing regret across the game tree's information sets, CFR converges toward a Nash equilibrium (Sutton 2018). In the context of heads-up limit Texas Hold'Em, the variant we focus on here, the game is already weakly solved. Bowling et al. showed that one can compute a CFR strategy that is guaranteed not to lose in expectation, regardless of the opponent

(Bowling 2015). Later work by Brown and Sandholm applied these principles to no-limit Hold’Em; their Libratus agent defeated top human professionals by augmenting a CFR blueprint with real-time endgame solving (Brown 2018). Because finding a Game Theoretically Optimal (GTO) baseline is a well-understood process, our paper doesn’t focus on achieving equilibrium. What we are interested in instead is whether a learning agent can deliberately deviate from that safe baseline to maximize profits against exploitable players.

Reinforcement Learning for Poker

Iterated CFR is computationally expensive, leading some researchers to try applying deep RL to poker without a CFR prior. For instance, AlphaHoldem demonstrated that an end-to-end RL framework driven by self-play can achieve competitive results without the massive overhead of CFR, utilizing a pseudo-Siamese architecture (Zhao 2022). When using value-based RL in this domain, overestimation bias in the Q-function is a known issue. To handle this, Zhang et al. combined a DQN model with Monte Carlo game search (Zhang 2020). Another major challenge is dealing with hidden information. Shi et al. addressed this in multiplayer poker by using an Actor-Critic setup under the Centralized Training, Decentralized Execution (CTDE) framework (Shi 2022). Their approach gives the critic access to all players’ cards during training, but restricts the actor to partial observations during actual execution.

Opponent Modeling and Exploitation

The idea of modeling an opponent to compute a specific best response has been around for a while. Ganzfried and Sandholm built an early game-theoretic system that updated a best response in real time by observing opponent action frequencies and combining that data with an equilibrium prior (Ganzfried 2011). Our architecture actually borrows from this frequency-tracking concept, which is why we feed an 8-dimensional hand summary into our opponent LSTM after each hand. On the neural network side, Li and Miikkulainen evolved LSTM models that were able to generalize to new opponents (Li 2017). They subsequently showed that these kinds of models could adapt mid-session, exploiting various playstyles much better than agents stuck playing a static Nash equilibrium (Li 2018).

Project Description

Limit Texas Hold’Em

In this paper, the game to which we are referring to as “poker” is Limit Texas Hold’Em. While less popular than the traditionally televised No Limit Texas Hold’Em, the for-

mer specifies fixed bet / raise sizes and a cap on the maximum number of actions that can be performed per street, which drastically reduces the state space. For the sake of “exploitation”, the game will also be “heads up”, with only two players (agent and opponent) competing against each other. This ensures that the game is zero sum.

Poker as a Partially Observable Markov Decision Process

Unlike games of perfect information such as chess, poker is a game of imperfect information. Each acting player only has access to a subset of information that does not include the identity of any opponents’ cards. From the player’s perspective, with identical looking observations that can map to different outcomes based on the hidden info, poker violates the Markov property, and cannot be modeled as a true Markov Decision Process.

We model poker as a Partially Observable Markov Decision Process (POMDP) defined by the tuple (S, A, O, P, R, Ω) , where the agent operates under incomplete information and must act based on observations rather than the full game state.

The true environment state $s \in S$ is fully specified by all information required to determine future transitions and rewards. This includes: all private hole cards for both players; the sequence of remaining cards in the deck; public community cards; the current pot size; the chip stack sizes for each player; and the current betting configuration (such as the amount required to call, the number of raises in the round, and player whose turn it is to act).

A subset of states $S_T \subset S$ corresponds to terminal states in which a hand has concluded, either due to a player folding or a showdown occurring following the river. In these states, no further actions are available, and the reward function assigns the pot to the winning player. Episodes are reset following terminal states.

At each time step t , the agent receives an observation $o_t \in O$, which consists of all of the same pieces of information in the true state, with the exception of hole cards belonging to the opponent, and the underlying state of the deck.

The action space A is discrete and corresponds to $A = \{\text{fold, call, raise, check}\}$ with legality constrained by the current betting configuration.

We designed opponents with fixed strategies within each episode. Under this assumption, the opponent’s policy does not change over time, and thus the environment dynamics and the transition function $T(s_{t+1} | s_t, a_t)$ remain stationary. Poker then becomes a single agent game.

Since the observation o_t is not Markovian, optimal decision making requires conditioning on the history of observations and actions $ht = (o_1, a_1, o_2, a_2, \dots, o_t)$.

LSTM Architecture

In a standard POMDP, a probability distribution of the true state s given an observation history h_t is maintained. This probability distribution is referred to as the belief state $b_t(s) = P(s | h_t)$. This belief state serves as a sufficient statistic for decision making, which restores the Markov property by capturing all relevant information from past interactions.

In practice, maintaining and updating an exact belief distribution is very computationally expensive, especially in complex games like poker. Our agent architecture replaces explicit belief tracking with a learned representation, where recurrent neural networks in the form of Long Short Term Memory (LSTM) encode the observation and interaction history into a hidden state $\sim s_t$. This hidden state functions as an implicit approximation of the belief state, enabling the agents to condition their policy and value estimates on a compressed summary of past information without performing explicit probabilistic inference.

While the underlying setting of poker remains a POMDP, our agent architecture reduces it to an approximate MDP by treating the learned hidden state $\sim s_t$ as a sufficient statistic for decision making. Therefore, we define the policy and value functions as $\pi(a_t | \sim s_t)$ and $V(\sim s_t)$.

Environment & Infrastructure

The game environment was simulated by RLCard, a library with support for a number of card games including Limit Hold’Em. The state representation given by this library is a 72-dimensional one-hot encoding of the game state, s . $s[0:52]$ represents the 52 cards in the deck (1 if present in either the current player’s hand or the board). $s[52:72]$ represents the raises that happened on each round ($s[52:57]$ for preflop, $s[57:62]$ for flop, $s[62:67]$ for turn, and $s[67:72]$ for river, with each round having 0, 1, 2, 3, or 4 raises). It is important to note that this does not constitute action history, but rather a compressed representation of aggression faced from the opponent, and should therefore not be discarded with regards to the aforementioned POMDP representation of poker.

We used the RLCard state representation, and also injected a few additional components to make the state representation a 77-dimensional one-hot encoding. $s[72:76]$ represents which of the actions (call = 0, raise = 1, fold = 2, check = 3) are currently legal, and $s[76:77]$ represents the current player’s position (small blind = 0, big blind = 1). We added these additional components to allow agents to consider only all legal actions and their position (which is a highly relevant component of poker) in selection.

Fixed Strategy Opponents

The real world archetypes of players are laid out below on a graph that characterizes each as a combination of the extent

of starting hands willing to be played and the amount of aggressive action (betting / raising) taken throughout the course of hands.

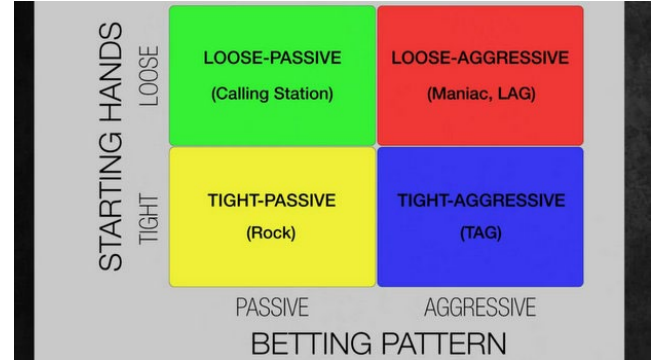


Figure 1: Player Archetype Map

We designed four fixed strategy opponents that represent extreme versions of these real world player archetypes. While not completely identical, these serve as simplistic models for which our agents can develop profitable policies.

The Calling Station represents a player who always calls or checks depending on which option is available. Optimal strategy to exploit this player consists of relentlessly value betting stronger than average hands and reducing bluffing frequency to zero. Pseudocode for this opponent is given below:

Calling Station Policy

Require: State s with legal actions $\mathcal{A}(s)$

Ensure: Action a

- 1: **if** CALL $\in \mathcal{A}(s)$ **then**
 - 2: $a \leftarrow$ CALL
 - 3: **else**
 - 4: $a \leftarrow$ CHECK
 - 5: **end if**
 - 6: **return** a
-

Figure 2: Calling Station Policy

The Maniac represents a player who always raises or calls depending on which option is available. Optimal strategy to exploit this player also consists of value betting stronger than average hands and reducing bluffing frequency to zero. Pseudocode for this opponent is given below:

Maniac Policy

Require: State s with legal actions $\mathcal{A}(s)$

Ensure: Action a

```

1: if RAISE  $\in \mathcal{A}(s)$  then
2:    $a \leftarrow$  RAISE
3: else
4:    $a \leftarrow$  CALL
5: end if
6: return  $a$ 

```

Figure 3: Maniac Policy

The Old Man Coffee represents a player who only raises or calls with the strongest hands (Aces, Kings, and Queens), and checks or folds all other hands. Optimal strategy to exploit this player is to play nearly every hand and steal the blinds, and fold when aggressive action is taken. Pseudocode for this opponent is given below:

Old Man Coffee Policy

Require: State s with hand ranks h and legal actions $\mathcal{A}(s)$

Ensure: Action a

```

1: Let  $h \leftarrow$  sorted ranks of player's hand
2: if  $h \in \{AA, KK, QQ\}$  then
3:   if RAISE  $\in \mathcal{A}(s)$  then
4:      $a \leftarrow$  RAISE
5:   else
6:      $a \leftarrow$  CALL
7:   end if
8: else if CHECK  $\in \mathcal{A}(s)$  then
9:    $a \leftarrow$  CHECK
10: else
11:    $a \leftarrow$  FOLD
12: end if
13: return  $a$ 

```

Figure 4: Old Man Coffee Policy

The Polarizing Player represents a player who raises or calls with their strongest and weakest hands, calls or checks with their middling hands, and checks or folds all other hands. Optimal strategy to exploit this player to value bet relentlessly with strong hands, bluff less, and play stronger than average hands to gain info on their potential hand strength. Pseudocode for this opponent is given below:

Polarizing Policy

Require: State s with hand (c_1, c_2) and legal actions $\mathcal{A}(s)$

Ensure: Action a

```

1: Let  $h \leftarrow$  sorted ranks of  $(c_1, c_2)$ 
2: if  $h \in \{AA, KK, QQ, AK\}$  or  $(h \in \{45, 56, 67, 78\} \wedge c_1, c_2 \text{ suited})$  then
3:   if RAISE  $\in \mathcal{A}(s)$  then
4:      $a \leftarrow$  RAISE
5:   else
6:      $a \leftarrow$  CALL
7:   end if
8: else if  $c_1, c_2$  form a pair or are suited or both are face cards then
9:   if CHECK  $\in \mathcal{A}(s)$  then
10:     $a \leftarrow$  CHECK
11:   else
12:     $a \leftarrow$  CALL
13:   end if
14: else if CHECK  $\in \mathcal{A}(s)$  then
15:    $a \leftarrow$  CHECK
16: else
17:    $a \leftarrow$  FOLD
18: end if
19: return  $a$ 

```

Figure 5: Polarizing Policy

DQN

Motivation and Hypothesis

Even with a reduced number of actions, Limit Hold’Em poker has an immense state space (3.16×10^{17} game states and 3.19×10^{14} decision points). Therefore, it was appropriate to develop a DQN to approximate the optimal action-value function $Q(s, a)$ for our environment. Rather than learning a tabular representation, the DQN uses a neural network parameterized by θ . Double DQN is also appropriate to mitigate overestimation bias, especially for poker, which has sparser rewards. We hypothesize that a Dueling Double DQN can learn strong exploitative strategies using a simpler and more stable training pipeline compared to policy gradient methods.

Modified State Representation

The state representation fed to the model was a 90-dimensional one-hot encoding of the game state. This state representation contains all of the same attributes as the 77-dimensional state encoding described above, as well as a few additional components. $s[0:72]$ remains the same as before. $s[72:82]$ represents the strength of the agent’s hand (royal flush = 10, straight flush = 9, four of a kind = 8, full house = 7, flush = 6, straight = 5, three of a kind = 4, two pair = 3, pair = 2, high card = 1). $s[82]$ represents whether or not a flush draw is present (four of one suit currently held, and only one more needed to make a flush). $s[83]$ represents whether or not a straight draw is present (four out of five numbers present in some consecutive run, and only one more needed to make a straight). $s[84]$ represents whether or not a full house draw is present (current hand is two pair or three of a kind). $s[85:89]$ and $s[89]$ remain the same legal actions and player position encoding as before. The intuition behind this choice was to remove the need for the model to learn both the semantic rules of poker along with proper exploitation strategy, especially given an already expansive

state space juxtaposed to a relatively limited amount of memory, compute, and time.

Feedforward Feature Encoder

The 90-dimensional state vector is passed through two fully connected layers with dimensions $90 \rightarrow 128$ and $128 \rightarrow 128$. The dimensional expansion allows the network to capture nonlinear interactions between different feature groups such as hand strength, board state, and action constraints (which are all very relevant in human decision making in poker as well). Multiple layers were used to capture higher level patterns.

LSTM Layer

The encoded features are processed by a single LSTM layer with hidden size 128, which operates over the sequence of decision points within a hand. This allows the model to incorporate betting history into its value estimates. Similar to the aforementioned LSTM architecture, this allows the model to maintain a hidden state that summarizes prior actions and observations and conditions action selection on this trajectory.

The LSTM state is reset at the start of each hand and does not persist across hands. This reflects the design goal of learning a best response to a fixed opponent rather than building a long term opponent model. Restricting recurrence to within-hand dynamics ensures that the network focuses on interpreting the current betting sequence, while also simplifying training by treating each hand as an independent sequence.

Dueling Heads

The output of the recurrent LSTM layer is passed into a dueling architecture that separates value estimation into two components: a value head $V(s)$ that represents the overall quality of the state, and an advantage head $A(s, a)$ that represents the relative benefit of each possible action in that state. These two quantities are combined to form the final action-value estimates using the equation below:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

Figure 6: Dueling Heads Action-Value Equation

In poker, many decision points involve actions with similar expected value. In such cases, it is inefficient for a standard Q-network to relearn the full value of the state independently for each action. Instead, the dueling formulation allows the shared state value to be learned once, while the advantage head captures only the small differences between actions. As a result, the network can learn more efficiently and produce more stable value estimates in situations where the relative ordering of actions is subtle but important. This normalization also stabilizes training.

Actor Critic

Motivation and Hypothesis

The DQN approach relies on developing a separate specialist agent for each opponent. The actor-critic agent instead aims to create a single generalist policy. The goal is to identify and exploit unknown opponents during a session without prior labels. We hypothesize that a recurrent opponent model can accumulate behavioral evidence across consecutive hands. This would allow the agent to transition from a safe baseline strategy early in a session to a targeted exploitative strategy once sufficient data is gathered. Three different architectures were explored sequentially, with each iteration addressing the failures of the previous one.

Architecture 1: Dual LSTM with Concatenation

This architecture separates temporal context using two recurrent modules that operate at different timescales. The 77-dimensional game state is passed through a feedforward trunk consisting of two fully connected layers ($77 \rightarrow 128 \rightarrow 64$, ReLU). The resulting encoding is processed by a Game LSTM with 64 hidden units. The state of this Game LSTM is reset at the start of every hand. This allows the model to handle partial observability within a single hand's betting sequence.

Opponent modeling is handled by a separate Opponent LSTM (8 $\rightarrow$ 32 hidden units) that persists across the entire session. Following each completed hand, an 8-dimensional summary vector is fed into this network. This vector encodes the opponent's normalized action frequencies, showdown occurrences, normalized payoff, and the number of betting rounds reached. The hidden state functions as a running behavioral profile.

The two streams are combined by concatenating the 64-dimensional game encoding and the 32-dimensional opponent context into a 96-dimensional vector. This vector is then passed into separate policy and value heads. An auxiliary predictor head (32 $\rightarrow$ 16 $\rightarrow$ 4, softmax) is attached to the opponent context and trained with mean squared error to predict cumulative action frequencies. The intuition behind this choice was to encourage the Opponent LSTM to encode useful behavioral patterns.

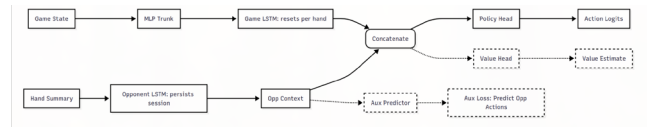


Figure 7: Actor Critic Architecture 1

Architecture 2: CTDE with FiLM Conditioning

The second architecture adopts the Centralized Training, Decentralized Execution (CTDE) paradigm (Srinivasan 2018). A critic is trained with access to both players' hole cards, represented as a 104-dimensional one-hot vector concatenated with the 77-dimensional game state. This gives the critic full knowledge of the better hand during training,

allowing for card-luck-aware value estimates. The actor only retains its own observation during inference.

The Game LSTM was removed for this version. The RLCard state representation already encodes raise-counts, which captures within-hand betting history, making the Game LSTM redundant. The actor is structured as a fully feedforward MLP trunk (77→128→128, ReLU), aligning with other strong poker agents like AlphaHoldem (Zhao 2022).

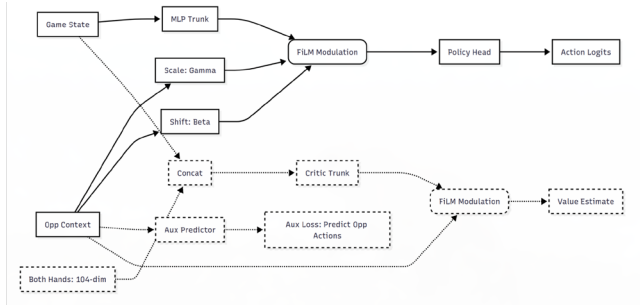
Concatenation is replaced with Feature-wise Linear Modulation (FiLM) (Perez 2018). The Opponent LSTM hidden state generates per-channel scale and shift parameters to multiplicatively modulate the MLP trunk's output:

$$x_{modulated} = \gamma(z_{opp}) \odot f(s_t) + \beta(z_{opp})$$

Figure 8: FiLM Equation

Here, γ and β are learned linear projections of the opponent context. This makes it structurally impossible for the network to ignore the opponent context. Every feature channel is scaled and shifted by opponent-dependent parameters. The MSE loss on cumulative action frequencies was replaced with a cross-entropy loss predicting the opponent's individual action at each decision point (He 2016). This provides a per-decision supervised gradient signal to the Opponent LSTM. The centralized critic uses an identical FiLM-conditioned trunk to the actor, but processes the expanded 181-dimensional input to produce a value estimate.

Figure 9: Actor Critic Architecture 2



Experiments

DQN Training

Training Setup

For the DQN agents, we went with a strict specialist approach. Four agents were paired up with the four fixed-strategy opponents, and we trained them exclusively against their assigned target for 50,000 hands (with a 200-hand replay buffer warmup before kicking off gradient updates). We weren't looking for a general policy here; the goal was deliberate overfitting to see if the agents could find the

sharpest possible exploit for one specific playing style. During these runs, the agents used an epsilon-greedy policy where we started epsilon at 1.0 and decayed it linearly down to 0.05. Instead of tying this decay to optimizer steps, we tracked it over 20,000 hands, since a complete hand is the most natural unit of experience in poker. Also, intermediate actions during a hand don't get any immediate reward, only the final terminal transition gets the payoff. Because of that, we set the discount factor to $\gamma=1.0$ so early betting decisions wouldn't get penalized just for happening sooner in the sequence.

Target and Policy

To keep overestimation bias in check (Van 2016), we used a Double DQN target where the online network handles action selection while the frozen target network actually evaluates it:

$$y_t = r_t + \gamma Q_{\text{target}}(s_{t+1}, \arg \max_a Q_{\text{online}}(s_{t+1}, a))$$

Figure 10: Double DQN Target

We synced the networks every 200 steps. This is particularly crucial because, again, rewards only appear at the end of the hand; without syncing, the bootstrap targets would just wildly chase the online network.

Loss Function

The network minimizes Huber loss between predicted Q-values and computed targets:

$$L_\delta(y - Q(s, a; \theta)) = \begin{cases} \frac{1}{2} (y - Q(s, a; \theta))^2, & \text{if } |y - Q(s, a; \theta)| \leq \delta \\ \delta (|y - Q(s, a; \theta)| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

Figure 11: Huber Loss Equation

We used Huber loss because it acts like MSE for small errors but switches to MAE for larger ones, making it handle outliers much better. This is great for poker because it doesn't overreact to the massive TD errors from sparse terminal rewards early in training.

Replay Buffer

Instead of storing individual shuffled transitions, our replay buffer holds entire hands as complete sequences. If we sampled isolated transitions, it would completely break the sequential structure that the LSTM relies on, making the recurrent hidden state essentially useless. The buffer has a capacity of 50,000 hands, and we sample mini-batches of 32 complete hands uniformly for each update.

Actor Critic Training

Training Setup

Training the actor-critic models required a completely different pipeline. Episodes consisted of 1,000 hands against one opponent, but we randomly swapped in a new opponent

at the start of every episode. We had to do this to force the Opponent LSTM to actually learn. If an agent just grinds against the same archetype, it has zero incentive to differentiate playing styles. Randomizing the opponents meant the network had to figure out what made each one unique, rather than just memorizing basic poker mechanics.

All variants were trained using PPO (Schulman 2017) to keep policy updates from getting too destructive:

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Figure 12: PPO CLIP Equation

For the first architecture, we stuck with Monte Carlo returns ($\gamma=1.0$). Since the player is only paid at the end of a hand, the return for every decision point in that hand is just the final payoff, making the advantage $A_t=R-V(s_t)$. The total loss looked like this:

$$\mathcal{L} = \mathcal{L}_{PPO} + 0.5\mathcal{L}_{value} + 0.5\mathcal{L}_{aux} - 0.01\mathcal{H}(\pi)$$

Figure 13: Total Loss Equation 1

In Architecture 2, Monte Carlo returns were swapped out for Generalized Advantage Estimation (Shuman 2016) ($\lambda=0.95$, $\gamma=1.0$). This setup bootstraps return estimates from the critic's value predictions at the boundary of each betting round instead of waiting for the terminal reward:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_{t+l} = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})$$

Figure 14: Generalized Advantage Estimation Equation

Because the centralized critic knows the hole cards, this approach yields advantage estimates with much lower variance. We also bumped the entropy coefficient up from 0.01 to 0.05, since higher entropy regularization has been shown to help PPO in games of imperfect information (Rudolph 2025). The new combined loss becomes:

$$\mathcal{L} = \mathcal{L}_{PPO} + 0.5\mathcal{L}_{value} + 0.3\mathcal{L}_{aux} - 0.05\mathcal{H}(\pi)$$

Figure 15: Total Loss Equation 2

All variants use the Adam optimizer with learning rate 3×10^{-4} , gradient clipping at 0.5, PPO clip $\epsilon=0.2$, and 4 PPO epochs per update.

Opponent Pool

The initial training pool only contained the four hardcoded archetypes. This actually caused a major problem. Because these four are extreme behavioral poles, the agent could face

wildly different playstyles in consecutive episodes, generating conflicting gradient signals that destabilized the learning. For example, an agent might learn to trap a Maniac in one episode, and then immediately face an Old Man Coffee (who folds 94% of the time) in the next, requiring the exact opposite strategy.

To fix this, Architecture 2 introduces a Parameterized Player. We set up continuous parameters (like VIP, preflop raise frequency, aggression factor, and fold-to-raise frequency) and sampled them uniformly across the four regions of the archetype grid. Training then used a 50/50 split of hardcoded and parameterized opponents. This filled in the gaps in the strategy space and smoothed out the gradient signals across episodes.

Training Curves and Architecture Evolution

To track adaptation, we mainly looked at the payoff split between the first half and the second half of each episode. If the agent is actually adapting mid-session, the second-half curve should rise clearly above the first-half curve as training goes on. If the lines are flat or overlapping, the policy is static.

With Architecture 1, the overall payoff essentially converged to zero. The adaptation curves never separated at any point during training, meaning they were statistically identical. This confirmed that the concatenation setup didn't give the actor any real incentive to use the opponent context. Card-luck variance was basically drowning out any gradient signal trying to reach the Opponent LSTM.

Architecture 2 improved the base policy quality quite a bit. Overall payoff hit +1.0 mbb/h, and the matchup against the maniac improved from -3.0 mbb/h to basically breakeven. However, the adaptation curves were still completely flat. The FiLM parameters converged to near-identity transforms, meaning the actor just settled into a static policy that was "good enough" against everything. The core issue here is that the PPO reward per hand is exactly the same whether the agent adapts at hand 50 or just plays a fixed strategy from hand 1. There was no objective pressure to adapt.

To deal with this, we tried a meta-batched training loop. We kept the Opponent LSTM in the computation graph for all 200 hands of a trial and optimized for the cumulative return. This creates a gradient path from the advantages late in the hand back to the early summaries where the opponent identity gets encoded (Duan 2016). After 500 meta-iterations, Architecture 2 showed better stability and tighter variance, but still zero separation in the adaptation curves. The base-policy gradient acting on the 43K actor parameters simply kept overwhelming the adaptation gradient acting on the 5K Opponent LSTM parameters. Ultimately, the Opponent

nent LSTM just kept learning whatever static representations were passively useful instead of anything that would trigger a mid-session adjustment.

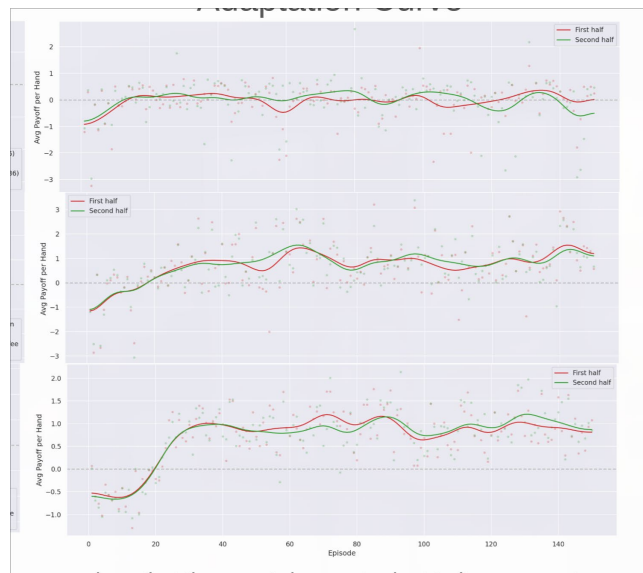


Figure 16: Training Graphs

Benchmarking Architecture

To figure out if any of these architectures actually worked, we measured everything in milli-big-blinds per hand (mbb/h). This normalizes the payoffs so a single big blind equals 1,000 mbb, which is pretty standard in the poker AI literature. Every trained agent had to play a 1,000-hand session against every opponent in the pool, and we repeated this across 20 random seeds just to smooth out the noise. By tracking the cumulative mbb/h hand-by-hand, we generated a mean performance curve with a shaded one-standard-deviation confidence band. Since cumulative mbb/h is basically a running average, the tail end of the curve at hand 1,000 gives us a solid estimate of the agent's true expected value. A curve settling above zero means we found a profitable exploit, near zero is breakeven, and negative means our agent is the one getting exploited. We intentionally ran a full cross-evaluation matrix, putting specialists against opponents they never trained on, to expose just how brittle the specialized models were compared to the generalist actor-critic. The purely random agent just served as our absolute baseline floor.

Results

Looking at the actual matchups, the results against the Calling Station (Figure 17) were exactly as expected. The DQN-calling agent completely crushed it, pulling in around +300 mbb/h simply by relentlessly value-betting against the passive caller. The AC variants didn't exploit much but held near breakeven, which showed up as consistently wider confidence bands. Old Man Coffee (Figure 18) ended up being

basically free money for almost everyone. DQN-omc led the pack at +350 mbb/h, but even DQN-polar hit +330 mbb/h, and somehow the random agent still managed +145 mbb/h. OMC folds so frequently that practically any strategy that aggressively steals blinds is going to print money; it's by far the most universally exploitable archetype in the pool. The Polarizing Player (Figure 19) gave us our first genuinely surprising result: DQN-calling scored +160 mbb/h despite never seeing this opponent during training. It turns out that an aggressive value-betting style transfers incredibly well against anyone with passive tendencies. In this specific matchup, AC-v2-meta stayed mostly stable with a slight upward trend, while the base AC-v2 actually deteriorated.

The Maniac matchup (Figure 20) is where things completely fell apart for the specialists. One would naturally expect DQN-maniac to dominate here, but it barely scraped by at -100 mbb/h, making it the absolute worst-performing specialist against its own target. The insane variance from the Maniac's constant unconditional aggression basically destroyed the value learning signal, even with 50,000 hands of direct training. Meanwhile, DQN-calling dominated yet again (+350 mbb/h) just by calling down the aggression and waiting to collect on strong hands. The non-calling DQN agents (omc and polar) collapsed entirely, dropping below -1,000 mbb/h because their blind-stealing habits got violently punished by the Maniac. The AC variants held near zero. The generalist policy absorbed all that aggression gracefully without blowing up like the out-of-element DQN specialists.

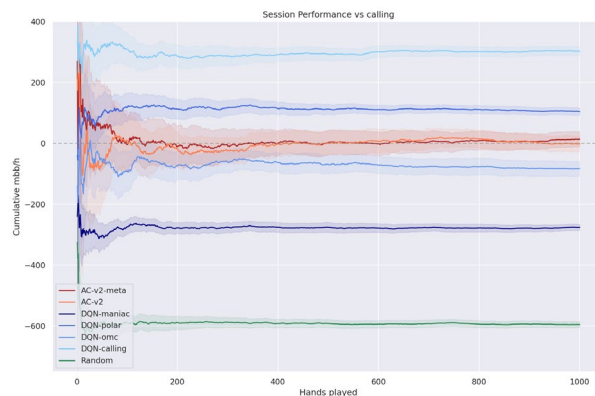


Figure 17: Performance Against Calling

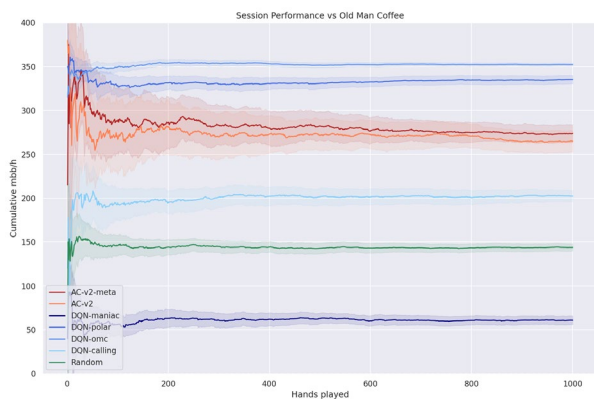


Figure 18: Performance Against Old Man Coffee

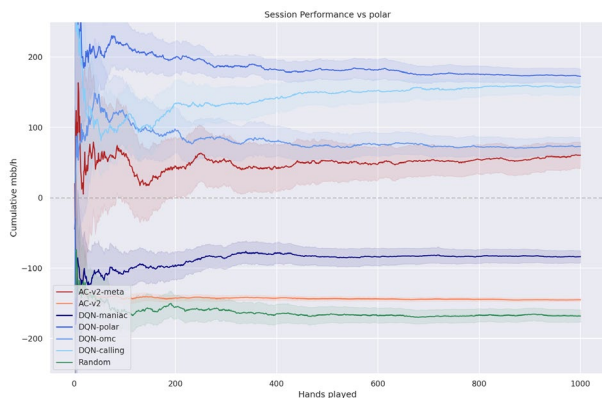


Figure 19: Performance Against Polar

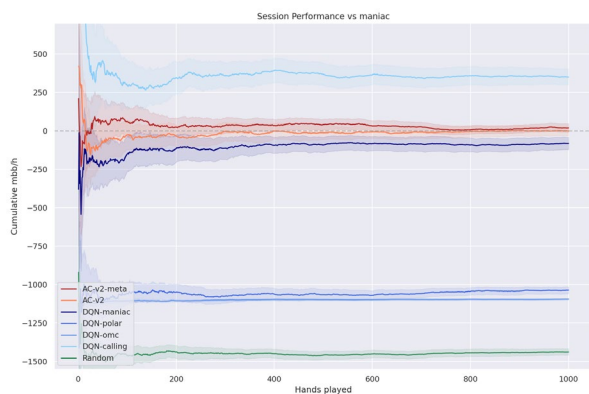


Figure 20: Performance Against Maniac

Conclusion

Ultimately, the results exposed a pretty harsh specialist-generalist tradeoff. We definitely proved that the DQN specialist approach works. Agents trained against a fixed archetype reliably learned highly profitable counter-strategies, scoring anywhere from +175 to +350 mbb/h against their targets.

The big surprise was just how well DQN-calling generalized to other opponents, suggesting that aggressive value-betting is just a great universal exploit against suboptimal, passive habits. However, the catastrophic failure of DQN-maniac highlighted a glaring limitation: value-based learning really struggles in extremely high-noise environments.

On the actor-critic side, the hypothesis only partially held up. The base policy was actually a success; it learned to play solid, reasonable poker across all archetypes without knowing who it was facing, completely avoiding the -1,000 mbb/h death spirals that the specialized DQN agents fell into. But the core objective, measurable, mid-session adaptation, completely failed across every single architecture we built. Whether we threw concatenation, FiLM conditioning, or meta-batched training at it, the Opponent LSTM just kept learning whatever static representation passively kept it out of trouble. The massive actor gradient simply overpowered the adaptation gradient every time, preventing any real on-the-fly strategic adjustments.

If we were to take this further, the obvious next step is completely decoupling those two objectives. We would likely need to pretrain the Opponent LSTM offline with a purely supervised loss, ensuring it actually learns a meaningful representation of the opponent before the massive actor gradient ever gets involved. Another promising route would be a hybrid setup: train the DQN specialists offline, but use a lightweight session-level classifier at inference time to actively switch between them. That way, we could get the massive win rates of the specialist with the safety net of a generalist.

References

- Bowling, M.; Burch, N.; Johanson, M.; and Tammelin, O. 2015. Heads-up Limit Hold'em Poker Is Solved. *Science* 347(6218):145–149.
- Brown, N.; and Sandholm, T. 2018. Superhuman AI for Heads-Up No-Limit Poker: Libratus Beats Top Professionals. *Science* 359(6374):418–424.
- Duan, Y.; Schulman, J.; Chen, X.; Bartlett, P. L.; Sutskever, I.; and Abbeel, P. 2016. RL²: Fast Reinforcement Learning via Slow Reinforcement Learning. *arXiv:1611.02779*.
- Ganzfried, S.; and Sandholm, T. 2011. Game Theory-Based Opponent Modeling in Large Imperfect-Information Games. In *Proceedings of the International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 533–540.
- He, H.; Boyd-Graber, J.; Daumé III, H.; and Eisenstein, J. 2016. Opponent Modeling in Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1804–1813.
- Li, X.; and Miikkulainen, R. 2017. Evolving Adaptive LSTM Poker Players for Effective Opponent Exploitation. In *AAAI Workshop on Computer Poker and Imperfect Information Games*.
- Li, X.; and Miikkulainen, R. 2018. Dynamic Adaptation and Opponent Exploitation in Computer Poker. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 3942–3951.
- Rudolph, M.; et al. 2025. PPO with High Entropy Regularization Outperforms Specialized Game-Theoretic Algorithms in Imperfect-Information Games. In *International Conference on Learning Representations (ICLR)*.
- Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.; and Abbeel, P. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *International Conference on Learning Representations (ICLR)*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
- Shi, D.; Guo, X.; Liu, Y.; and Fan, W. 2022. Optimal Policy of Multiplayer Poker via Actor-Critic Reinforcement Learning. *Entropy* 24(6):774.
- Srinivasan, S.; Lanctot, M.; Zambaldi, V.; Pérolat, J.; Tuyls, K.; Munos, R.; and Bowling, M. 2018. Actor-Critic Policy Optimization in Partially Observable Multi-agent Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31.
- Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.
- van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep Reinforcement Learning with Double Q-Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2094–2100.
- Zhang, X. C.; and Li, Y. 2020. A Texas Hold'em Decision Model Based on Reinforcement Learning. In *2020 Chinese Control and Decision Conference (CCDC)*, 1825–1829. IEEE.
- Zhao, E.; Yan, R.; Li, J.; Li, K.; and Xing, J. 2022. AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 4689–4697.